

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3



Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1

Testing as an Engineering Activity – Structured Test Design

Bloom's L3

Question: A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

1 CONCEPT

Software testing is not a random 'try-and-see' activity; it is an engineering discipline. Like any engineering activity it is planned, uses defined techniques, produces documented work products, is measurable and is repeatable. Structured test design means deriving test cases systematically from requirements and design using techniques such as Equivalence Partitioning, Boundary Value Analysis and Decision Tables, instead of relying on the tester's mood or memory.

2 APPLICATION TO THE SCENARIO

- **Analyse requirements:** Identify what must be verified and list testable conditions for every requirement.
- **Define a test plan:** Fix scope, objectives, resources, schedule, entry/exit criteria and risks so every team member follows the same approach.
- **Design test cases with techniques:** Apply EP, BVA, decision tables and state-transition models to cover valid, invalid and boundary situations.
- **Maintain traceability:** Map each test case to a requirement (Requirement Traceability Matrix) so that no requirement is left untested.
- **Execute, log and measure:** Run tests with defined data, log defects in a tracker and measure coverage and defect detection percentage.
- **Review and improve:** Analyse results, update the test suite and feed lessons back into the process.

3 DETAILED EXPLANATION

- **Better coverage:** Systematic derivation makes sure all requirements and input classes are exercised, so fewer defects are missed.
- **Repeatability:** The same test cases give the same results in every cycle, so results are consistent and comparable.
- **Efficiency:** Techniques such as EP reduce thousands of possible inputs to a small, high-yield set of test cases.
- **Measurability:** Metrics (requirement coverage, defects per test, detection %) show whether quality is improving.

4 EXAMPLE

Example: For a login page, random testing may try only two or three usernames. Structured design partitions inputs into valid user, invalid user, blank, locked account and special characters, and applies BVA to a password length rule of 8–16 characters (tests with 7, 8, 9, 15, 16, 17 characters). This catches defects that random testing would miss.

Conclusion: Treating testing as an engineering activity replaces inconsistency with planned, traceable and measurable testing, which directly improves defect detection and software quality.

Question: A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

1 CONCEPT

A testing process is an organised set of phases and activities – test planning, test design, test environment set-up, test execution, defect reporting and test closure – each with defined inputs, outputs, roles and exit criteria. In real-time, high-risk systems such as banking, where one failure can mean financial loss and regulatory penalties, a defined process is essential.

2 APPLICATION TO THE SCENARIO

- **Planning:** Define scope (fund transfer, deposits, interest calculation, ATM, net banking), risk areas and acceptance criteria.
- **Design:** Create test cases for normal flows, failures, boundary amounts, concurrency and security.
- **Environment and data:** Prepare a production-like environment with masked data and stubbed external payment gateways.
- **Execution and reporting:** Execute tests, log defects with severity/priority and track them to closure.
- **Closure:** Check exit criteria, publish test summary and record lessons learned.

3 DETAILED EXPLANATION

- **Fewer production failures:** Planned coverage of critical paths such as double debit or failed rollback reduces live incidents.
- **Predictability:** Everyone knows what to test, when and how, so schedules and quality become predictable.
- **Compliance and audit:** Documented test evidence supports regulatory audits (e.g. RBI/PCI-DSS style requirements).
- **Continuous improvement:** Metrics and closure reports help in improving the next release.
- **Reduced cost:** Defects are found before go-live, when fixing is cheaper and less risky.

4 EXAMPLE

Example: Without a process, a funds-transfer module was released in which a network timeout debited the sender but never credited the receiver. With a defined process, a test case 'timeout during transfer' is designed from the risk analysis, run in the test environment, the defect is found, fixed and verified, and a regression test is added so the failure never returns.

Conclusion: A standardised testing process makes quality systematic rather than accidental, which is exactly what reliable real-time banking software needs.

Question: During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

1 CONCEPT

Testing should be performed as a structured process that runs in parallel with every SDLC phase, not as a single activity at the end. In the V-model, every development phase has a corresponding test level: requirements ↔ acceptance testing, system design ↔ system testing, high-level design ↔ integration testing and detailed design ↔ unit testing.

2 APPLICATION TO THE SCENARIO

- **Requirements phase:** Review requirements, define acceptance criteria and prepare the acceptance test plan.
- **Design phase:** Prepare system and integration test plans and review design for testability.
- **Coding phase:** Developers perform unit testing and code reviews; static analysis is run.
- **Integration:** Test interfaces between cart, catalogue, payment and order modules using top-down/bottom-up or incremental integration.
- **System and acceptance:** Validate end-to-end flows (search → cart → payment → delivery) and run user acceptance tests.
- **Quality gates:** Do not move to the next phase until entry/exit criteria of the current phase are met.

3 DETAILED EXPLANATION

4 Table: V-Model mapping of development and testing activities

SDLC Phase	Testing Activity	Output
Requirement analysis	Requirement review, acceptance test planning	Acceptance test plan
System design	System test planning, design review	System test cases
High-level design	Integration test design	Integration test cases
Coding	Unit testing, code review	Unit test report
Integration / System	Integration and system testing	Defect reports, test summary
Release	Acceptance testing, regression	Sign-off report

- **Early defect detection:** Interface and logic defects are caught at the level where they are introduced.
- **Discipline:** Entry/exit criteria, standard templates and reviews remove inconsistency across phases.
- **Traceability:** Each requirement is traced to design, code and test cases, so integration and system gaps are visible.
- **Predictable quality:** Defect trends per phase give management clear information for release decisions.

5 EXAMPLE

Example: A mismatch between the cart module's price (in paise) and the payment module's price (in rupees) is found in integration testing because interface test cases were planned during design; without this process it would appear as a wrong-amount charge in production.

Conclusion: Process discipline ensures that testing is planned, phase-aligned and gated, which reduces integration and system-level defects and gives better quality outcomes.

Question: A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

1 CONCEPT

Error (mistake): a human action that produces an incorrect result, e.g. a developer misunderstands a rule. **Defect (fault/bug):** the flaw in the code, design or document caused by that error. **Failure:** the deviation of the system's observed behaviour from expected behaviour when the defective code is executed. The chain is Error → Defect → Failure.

2 APPLICATION TO THE SCENARIO

- **Observation:** The expected output ('Login successful' for a valid 8-character password) differs from the actual output ('Invalid password').
- **Classification:** This mismatch seen during validation is a failure.
- **Cause analysis:** On inspecting code, the condition is written as `password.length > 8` instead of `>= 8`; this incorrect line is the defect.
- **Origin:** The developer's misreading of the requirement 'minimum 8 characters' is the error.

3 DETAILED EXPLANATION

Table: Relationship between the terms for the login issue

Term	Meaning	In this login scenario
Error	Human mistake	Developer misunderstands the 'minimum 8 characters' rule
Defect / Fault / Bug	Flaw in the software artefact	Wrong operator '>' in the validation code
Failure	Observed incorrect behaviour at run time	Valid 8-character password is rejected

- **Not every error leads to a failure:** If the code path is never executed, the defect stays hidden.
- **A defect can cause many failures:** One wrong operator affects every user with an 8-character password.
- **Testers observe failures; developers locate defects:** Debugging traces the failure back to the defect and error.

4 EXAMPLE

Example: Error: developer assumes 'more than 8'. Defect: `if(len > 8)`. Failure: user with password 'Pass@123' (8 chars) cannot log in.

Conclusion: The issue reported by the team is a failure caused by a defect that originated from a developer error; correct classification helps assign it for the right corrective action.

Question: A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

1 CONCEPT

The principle of early testing states that testing activities should start as early as possible in the software life cycle, from the requirements phase itself, and should be focused on defined objectives. It includes static techniques (reviews, walkthroughs, inspections) as well as early dynamic testing (unit and integration testing).

2 APPLICATION TO THE SCENARIO

- **Review requirements:** Testers and developers inspect the SRS for ambiguity, missing or conflicting statements.
- **Review design:** Check design documents for testability and interface issues.
- **Early test design:** Write test cases while requirements are being finalised, which itself exposes gaps.
- **Unit and static testing:** Use unit tests, code review and static analysis tools before code is integrated.
- **Continuous feedback:** Run frequent builds with quick smoke tests so defects are reported soon after being introduced.

3 DETAILED EXPLANATION

- **Lower cost:** The cost of fixing a defect rises sharply as it moves from requirements to design, coding, testing and production.
- **Prevents defect propagation:** A wrong requirement fixed early does not become wrong design, wrong code and wrong tests.
- **Saves time:** Less rework in later stages shortens the schedule.
- **Better focus:** Test effort is directed to the risky and critical areas identified early.

4 EXAMPLE

Example: A requirement says 'Discount will be applied for loyal customers'. In a review, the tester asks 'Who is a loyal customer and what is the discount?' Clarifying it costs a few minutes; discovering the issue after release would need rework of design, code, test cases and customer data.

Conclusion: Early testing finds defects when they are cheapest to remove, which directly raises defect detection efficiency and lowers total project cost.

Question: During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

1 CONCEPT

In the requirement phase, the tester acts as the first 'user' of the specification. Through requirement reviews and testability analysis the tester checks that each requirement is clear, complete, consistent, unambiguous, feasible and testable. Early testing involvement means testers join the project from the beginning and not only when code is ready.

2 APPLICATION TO THE SCENARIO

- **Review requirements:** Use checklists to detect ambiguity ('fast', 'user friendly', 'adequate'), missing conditions and conflicts.
- **Raise clarification queries:** E.g. 'Dose must be adjusted for elderly patients' – what is the age limit and the adjustment rule?
- **Define acceptance criteria:** Write measurable criteria for every requirement.
- **Identify risks:** Flag high-risk functions such as medication dosage, patient identity and data privacy.
- **Prepare test conditions:** Derive test ideas and build the Requirement Traceability Matrix (RTM).
- **Plan test strategy:** Estimate effort, tools and environment based on the requirements.

3 DETAILED EXPLANATION

- **Ambiguities are removed before they become defects:** Requirement defects are the most expensive kind because they spread to design, code and tests.
- **Testability improves:** Requirements that are measurable are easy to verify.
- **Shared understanding:** Developers, testers and business stakeholders interpret requirements the same way.
- **Safety and compliance:** In healthcare, early identification of critical scenarios protects patient safety.

4 EXAMPLE

Example: In a medication module, a tester notes that the requirement does not specify behaviour when a drug allergy is recorded. Raising this query results in a new requirement to block prescription and display a warning – a defect avoided before a single line of code was written.

Conclusion: Early involvement of testers in requirement analysis prevents ambiguity-driven defects, reduces rework and improves the quality of safety-critical healthcare software.

Question: A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

1 CONCEPT

Defects can be injected at every phase of the software life cycle – requirements, design, coding and even testing/documentation – but studies show that a large share originates in requirements and design. Unclear, incomplete or ambiguous requirements lead to wrong assumptions that propagate downstream.

2 APPLICATION TO THE SCENARIO

3 Requirement phase: Requirement stated as 'Overtime is paid at 1.5 times the normal rate' without defining when overtime starts (daily 8 hours or weekly 40 hours).

- **Design phase:** Designer assumes weekly overtime and designs calculation logic accordingly.
- **Coding phase:** Developer implements the design faithfully, creating a defect relative to the business expectation.
- **Testing phase:** Testers who rely on the same unclear requirement write matching but wrong expected results, so the defect escapes.
- **Operation:** Employees receive incorrect salaries – a failure.

4 DETAILED EXPLANATION

Table: Defect injection and propagation across the life cycle

Phase	Typical defect source	Payroll example
Requirements	Ambiguity, omissions, conflicts	Overtime rule not defined clearly
Design	Wrong interpretation, missing interface	Weekly overtime logic designed
Coding	Logic, data, boundary mistakes	Rounding applied before tax calculation
Testing	Incorrect expected results, poor coverage	Test data does not include 9-hour days

- **Write SMART requirements:** Specific, measurable, achievable, relevant and testable statements.
- **Use examples and acceptance criteria:** 'If employee works 10 hours on a day, then 2 hours are paid at 1.5×'.
- **Review and inspect:** Joint reviews by business, developers and testers remove ambiguity.
- **Maintain a glossary and RTM:** Common terms and traceability stop assumptions from creeping in.
- **Change control:** Every requirement change is documented and re-assessed.

5 EXAMPLE

Example: After clarifying the overtime rule and adding three worked examples to the SRS, the payroll calculation test cases had unambiguous expected results and the incorrect salary defects stopped appearing.

Conclusion: Most payroll defects originate in unclear requirements; clear, reviewed and testable requirements prevent defects at their source.

Question: A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

1 CONCEPT

A boundary defect occurs when the software behaves incorrectly at the limits of an input range, typically because of a wrong relational operator ($<$ instead of $<=$), an off-by-one error or missing validation. As per Burnstein's defect classes, it is a code defect – specifically a control, logic and sequence / algorithmic defect with a data-validation aspect. From a testing viewpoint it is also a test-design (coverage) defect because the boundary was not tested.

2 APPLICATION TO THE SCENARIO

- **Classify by origin:** Coding defect (logic/data validation).
- **Classify by type:** Functional defect revealed by boundary input.
- **Classify by severity:** High or critical, because the system crashes.
- **Justify:** The crash happens only at the minimum, maximum or just beyond them – precisely where programmers commonly make off-by-one mistakes – and the same defect would not appear for ordinary values.

3 DETAILED EXPLANATION

Table: BVA test values for an input field that accepts quantity 1–100

Boundary	Test value	Expected result
Below minimum	0	Rejected with error message
Minimum	1	Accepted
Just above minimum	2	Accepted
Nominal	50	Accepted
Just below maximum	99	Accepted
Maximum	100	Accepted
Above maximum	101	Rejected with error message

- **How BVA prevents the issue:** BVA selects test values at, just below and just above each boundary, so the faulty condition would have been executed and the crash found during testing.
- **Combined with equivalence partitioning:** One representative value per partition plus boundary values gives high defect yield with few test cases.

4 EXAMPLE

Example: If the code was written as `if(qty < 100)` instead of `qty <= 100`, the test with 100 would fail, revealing the defect immediately; entering only '50' would never have exposed it.

Conclusion: The crash is a boundary-related coding (logic) defect of high severity that proper BVA test cases at and around the limits would have detected before release.

Question: A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

1 CONCEPT

A defect repository is a central database in which every defect and its attributes (ID, module, description, severity, priority, phase injected, phase detected, root cause, status, resolution) are recorded. It is a valuable historical knowledge base for test teams.

2 APPLICATION TO THE SCENARIO

- **Analyse defect patterns:** Group defects by module, type and root cause to find weak areas (e.g. validation and boundary defects).
- **Design new test cases:** Add test cases specifically for the defect types that occurred earlier.
- **Prepare checklists:** Build error-guessing checklists from past defects for reviewers and testers.
- **Prioritise testing:** Give more test effort to defect-prone modules.
- **Create regression tests:** Convert every fixed defect into a regression test case.
- **Feed back to development:** Share trends so that developers avoid similar mistakes.

3 DETAILED EXPLANATION

- **Reduces repeated defects:** Known weaknesses are tested in every release.
- **Improves test effectiveness:** Test cases are based on evidence and not on assumptions.
- **Supports metrics:** Defect density, defect removal efficiency and trend charts become available.
- **Preserves knowledge:** New team members learn from the history of the product.

4 EXAMPLE

Example: Repository analysis shows 18 of 60 defects were date-validation errors (leap year, month end). The team adds a date-validation test set and a review checklist item; in the next release, such defects drop sharply.

Conclusion: When actively used, the defect repository converts past failures into better test cases, which prevents recurrence and steadily raises software quality.

Question: An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

1 CONCEPT

A functional defect is a deviation in which the software does not perform a function as specified in the requirements. In the ATM system, the incorrect handling of withdrawal transaction logic (for example, deducting the balance but not dispensing cash, or dispensing without updating the balance) means that the withdrawal feature does not behave as required – hence a functional defect.

2 APPLICATION TO THE SCENARIO

- **Identification:** Expected: balance is checked, cash is dispensed, balance is reduced exactly once and a receipt is produced. Actual: transaction fails or is processed incorrectly.
- **Classification:** Functional (logic/algorithmic) defect of critical severity because it impacts money and customer trust.
- **Why it escaped:** Test cases did not cover transaction sequences, failure points (cash jam, power or network failure) or rollback behaviour.

3 DETAILED EXPLANATION

- **Review requirements and design:** Use review of the transaction flow and state diagram to find missing conditions.
- **State-transition testing:** Model states (card inserted, PIN verified, amount entered, cash dispensed, completed) and test valid and invalid transitions.
- **Decision tables:** Cover combinations such as sufficient balance / daily limit / ATM cash available.
- **Negative and fault-injection tests:** Simulate timeouts, hardware errors and interruptions to verify rollback (ACID behaviour).
- **Unit and integration testing:** Test transaction logic with stubs for the bank host and cash dispenser.
- **Code reviews and regression tests:** Review the transaction code and automate regression tests for all withdrawal rules.

4 EXAMPLE

Example: Test case: balance ₹5,000, withdraw ₹5,000 (exact balance) → success; withdraw ₹5,001 → declined; network fails after debit → transaction reversed and balance restored. Such cases would have exposed the defect.

Conclusion: The ATM failure is a critical functional defect that could have been prevented by requirement review, state-based and decision-table test design and thorough negative and rollback testing.

Question: A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

1 CONCEPT

Defect clustering is a testing principle which states that a small number of modules usually contain most of the defects (the Pareto or 80/20 rule: about 80% of defects come from about 20% of modules). Clusters arise from complexity, frequent change, new technology, poor design or inexperienced developers.

2 APPLICATION TO THE SCENARIO

- **Collect data:** Count defects per module from the defect repository and earlier releases.
- **Rank modules:** Draw a Pareto chart and identify the top defect-prone modules.
- **Prioritise:** Combine defect density with business criticality and change frequency (risk-based testing).
- **Allocate effort:** Give deeper test design, more test cycles, code review and senior testers to the clusters.
- **Re-evaluate:** Update the clusters after each cycle, because clusters move as defects are fixed.

3 DETAILED EXPLANATION

Table: Illustrative defect distribution in a 10-module application (100 defects)

Module	Defects	% of total	Testing priority
Payment	40	40%	Very high
Order Processing	30	30%	Very high
Cart	10	10%	High
Other 7 modules (combined)	20	20%	Normal / Low

- **Efficiency:** Most defects are found with a fraction of the effort because the effort is concentrated where defects live.
- **Better resource use:** Limited time and testers are used where they give the highest return.
- **Risk reduction:** Critical clusters are tested more deeply before release.
- **Caution:** Do not ignore other modules completely, and avoid the pesticide paradox by refreshing test cases regularly.

4 EXAMPLE

Example: In the table, two modules hold 70% of defects, so the team assigns more test cases, more exploratory testing and detailed code review to Payment and Order Processing.

Conclusion: Using defect clustering allows the team to focus on high-yield areas first, increasing defect detection per unit of effort.

Q12

Defect Classification – Performance Defect under Heavy Load

Bloom's L3

Question: A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

1 CONCEPT

Defect classification is the process of grouping defects according to attributes such as type, origin, severity, priority, status and root cause. Slow response under heavy user load is a non-functional defect – specifically a performance (load/scalability) defect.

2 APPLICATION TO THE SCENARIO

- **By type:** Non-functional – performance / scalability defect.
- **By severity:** Major to critical, because it affects availability and user experience for many users.
- **By priority:** High, because the business impact is immediate during peak traffic.
- **By origin:** Usually a design or code defect (inefficient queries, no caching, thread or connection-pool limits, insufficient hardware sizing).
- **By detection method:** Found through load, stress or endurance testing or production monitoring.

3 DETAILED EXPLANATION

Table: Defect classification of the slow-performance issue

Classification criterion	Category assigned	Reason
Nature	Non-functional (performance)	Features work but response time is unacceptable
Severity	Major / Critical	Affects many users under load
Priority	High	Needs a fix before peak season
Origin phase	Design / Coding	Inefficient design, unoptimised queries
Test type to verify	Load, stress, scalability	Simulates concurrent users

- **Right owner:** Classified defects reach the proper team (database, network, application) quickly.
- **Correct prioritisation:** Severity and priority guide the fix order.
- **Root cause analysis:** Origin and type reveal systematic weaknesses such as missing performance requirements.
- **Trend analysis:** Classification statistics show which defect types repeat and what to prevent.
- **Right verification:** Fixes are re-tested using appropriate performance tests.

4 EXAMPLE

Example: Response time rises from 2 seconds to 14 seconds at 1,000 concurrent users. It is logged as a high-severity performance defect, traced to a missing database index, fixed and verified with a repeat load test.

Conclusion: Proper classification turns an unclear complaint ('the site is slow') into an actionable, prioritised and traceable defect that is resolved faster and prevented in future.

Question: A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

1 CONCEPT

Regression testing re-executes tests after changes to confirm that existing functionality still works and that new changes have not introduced new defects. Historical defect data – which modules failed, how often, why and what was changed – is the best guide for deciding what to include in the regression suite.

2 APPLICATION TO THE SCENARIO

- **Identify high-risk areas:** Select modules with high defect density or frequent reopen/regression defects.
- **Add tests from fixed defects:** Every previously fixed defect should have a test case in the regression suite to detect reintroduction.
- **Impact analysis:** Use defect links and change history to find modules affected by a new change.
- **Prioritise the suite:** Order tests by risk: critical business flows and defect-prone modules first (smoke → sanity → full regression).
- **Remove redundancy:** Retire test cases that never find defects and replace them with new, defect-driven ones.
- **Automate:** Automate stable, frequently run regression tests for faster feedback.

3 DETAILED EXPLANATION

- **Better coverage:** Areas where defects actually occurred are covered, not just areas that seem important.
- **Efficiency:** Test time is spent on high-yield tests instead of re-running everything blindly.
- **Fewer escapes:** Reappearance of old defects is detected immediately.
- **Optimised effort:** Selection and prioritisation fit into tight release timelines.

4 EXAMPLE

Example: Defect history shows that tax calculation broke after three of the last five releases. The team adds those three defect scenarios, plus tests for related discount logic, to the automated regression pack and runs them on every build.

Conclusion: Historical defect data makes regression testing risk-based, focused and efficient, increasing coverage of vulnerable areas while saving effort.

Q14

Role of the Tester in Improving Collaboration and Reducing Integration Defects

Bloom's L3

Question: During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

1 CONCEPT

A tester is not only a defect finder but also a quality advocate and communication link between business, developers and operations. Poor communication causes misunderstanding of requirements and interfaces, and these surface as integration defects. The tester minimises them through early, continuous collaboration.

2 APPLICATION TO THE SCENARIO

- **Join requirement and design reviews:** Raise ambiguities and interface mismatches early.
- **Share test cases and acceptance criteria:** Developers understand expected behaviour before coding.
- **Interface/contract verification:** Confirm data formats, units, error codes and protocols between modules.
- **Clear defect reporting:** Write reproducible, objective defect reports with steps, data and evidence, avoiding blame.
- **Regular communication:** Take part in stand-ups, defect triage meetings and build/integration planning.
- **Common tools:** Use one shared tracker and documentation so that information is visible to all.

3 DETAILED EXPLANATION

- **Shared understanding:** Developers and testers interpret requirements and interfaces in the same way.
- **Early defect discovery:** Problems in specifications and designs are caught before integration.
- **Less rework and friction:** Early clarification replaces late arguments about 'bug or feature'.
- **Smoother integration:** Interface problems are resolved before modules are combined, so integration builds are more stable.

4 EXAMPLE

Example: During design review the tester notices that the order module sends date as DD-MM-YYYY while the billing module expects YYYY-MM-DD. Fixing it on paper costs minutes; it would otherwise appear as many integration failures.

Conclusion: By collaborating from the start, a tester reduces misunderstandings and integration defects and helps the whole team deliver higher quality faster.

Question: A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

1 CONCEPT

Defect prevention is a proactive approach that identifies the root causes of defects and changes the process so that similar defects do not occur again. It moves the focus from 'finding and fixing' to 'avoiding' defects, and is a key practice of mature processes (e.g. CMMI/TMMi).

2 APPLICATION TO THE SCENARIO

- **Collect and classify:** Gather defect data from all releases with type, origin and cause.
- **Root cause analysis:** Use Pareto analysis, fishbone diagram and 5-Whys to find why defects recur.
- **Define preventive actions:** Introduce coding standards, checklists, peer reviews, static analysis, training and better requirement templates.
- **Implement and monitor:** Embed actions into the process and track the defect trend release by release.
- **Share lessons:** Hold retrospectives and update the knowledge base.

3 DETAILED EXPLANATION

Table: From recurring defect to preventive action

Recurring defect	Root cause	Preventive action
Null-pointer crashes	No input/null checks	Coding standard and static analysis rule
Wrong calculations	Ambiguous requirements	Requirement review with worked examples
Same UI defects	No UI checklist	UI review checklist before release
Old defects return	No regression test	Add every fixed defect to regression suite

- **Fewer defects injected:** Fix the cause, not the symptom, so the number of defects falls.
- **Lower cost:** Less rework, retesting and production support.
- **Higher reliability:** Stable, predictable releases improve customer confidence.
- **Process maturity:** Learning becomes institutionalised and not dependent on individuals.

4 EXAMPLE

Example: After analysis showed 35% of recurring defects came from missing input validation, the team introduced a validation checklist and automated static checks; validation defects dropped by more than half in two releases.

Conclusion: Defect prevention attacks the root causes of recurring problems and therefore gives a long-term improvement in software quality.

Q16

Redesigning the Mobile App Test Strategy Using Testing Principles

Bloom's L3

Question: A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

1 CONCEPT

The seven testing principles are: (1) testing shows the presence of defects, (2) exhaustive testing is impossible, (3) early testing, (4) defect clustering, (5) pesticide paradox, (6) testing is context dependent, and (7) absence-of-errors fallacy. A test strategy built on these principles gives a standard, risk-focused approach.

2 APPLICATION TO THE SCENARIO

- **Exhaustive testing impossible → risk-based selection:** Choose a device/OS matrix from usage analytics and prioritise critical flows such as login, payment and sync.
- **Early testing:** Review designs, run unit tests and test every build with smoke tests in CI.
- **Defect clustering:** Focus on crash-prone screens and modules from crash analytics.
- **Pesticide paradox:** Refresh and extend test cases each release; add exploratory sessions.
- **Context dependence:** Cover mobile-specific factors: interruptions (calls, notifications), low memory, battery, poor/no network, orientation, permissions.
- **Standardise:** Common test case templates, defect severity guidelines, exit criteria and automated regression for stable flows.
- **Absence-of-errors fallacy:** Include usability and performance tests so the app is also useful, not only defect-free.

3 DETAILED EXPLANATION

- **Consistency:** All testers follow the same documented strategy, so results are comparable.
- **Reliability:** Crashes caused by memory, network and device differences are found before release.
- **Efficiency:** Risk-based selection and automation give maximum coverage in limited time.
- **Continuous learning:** Production crash data feeds back into new tests.

4 EXAMPLE

Example: Crash analytics show most crashes happen on low-memory Android devices during image upload. The new strategy adds low-memory and interrupted-upload tests on those devices to every regression cycle.

Conclusion: Applying testing principles produces a standard, risk-based strategy that makes the mobile application more stable and reliable.

Question: A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

1 CONCEPT [Conceptual Understanding]

Testing is the process of executing software to detect failures and evaluate quality; it is performed mainly by testers and it shows that defects exist. Debugging is the developer's activity of locating the cause of a reported failure, correcting the defect and verifying the fix. Testing finds symptoms; debugging finds and removes causes.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Scenario:** In an online-shopping app the order total shows ₹1,200 instead of ₹1,080 after a 10% coupon.
- **Testing role:** The tester executes the coupon test case, observes the failure, records steps, data and screenshots, and raises a defect.
- **Debugging role:** The developer reproduces the problem, traces the code, finds that discount is calculated on the pre-tax amount and applied twice/not at all, and corrects it.
- **Re-testing:** The tester re-executes the failed test and the related regression tests to confirm the fix.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Difference between testing and debugging

Aspect	Testing	Debugging
Purpose	Detect failures and assess quality	Locate and remove the cause of a failure
Performed by	Testers (also developers)	Developers
Starting point	Test cases and requirements	A reported failure/defect
Outcome	Defect reports, test results	Corrected code
Planned?	Planned, scheduled, repeatable	Unplanned, reactive
Can be automated?	Largely yes	Mostly manual and analytical
Knowledge of code	Not always required (black box)	Detailed code knowledge required

- **Together they improve quality:** Testing makes defects visible; debugging removes them; re-testing confirms the removal.
- **Separation avoids delays:** When roles are clear, testers do not spend time fixing and developers do not spend time hunting defects without proper reports.

4 EXAMPLE [Application & Clarity]

Example: A good defect report from testing (steps, data, expected vs actual result) shortens the debugging time significantly.

Conclusion: Testing and debugging are distinct but complementary activities; clarity between them speeds up defect resolution and improves software quality.

Question: An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

1 CONCEPT [Conceptual Understanding]

Cross-browser testing verifies that a web application functions and appears consistently across different browsers (Chrome, Firefox, Safari, Edge), versions, operating systems and devices. It applies the testing principles that exhaustive testing is impossible and that testing is context dependent: coverage must be chosen on the basis of risk and usage.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Define the compatibility matrix:** Select browser–OS–device combinations from analytics (e.g. Chrome/Windows, Safari/iOS, Chrome/Android, Edge, Firefox).
- **Prioritise features:** Test critical journeys first: search, product page, cart, checkout and payment.
- **Test layers:** Functional behaviour, UI layout and CSS rendering, JavaScript, forms and validation, cookies/local storage, fonts, media, responsiveness.
- **Use tools:** Cloud grids such as BrowserStack/Selenium Grid plus real devices; visual-regression tools for screenshot comparison.
- **Automate:** Run the same automated functional suite across browsers on every build.
- **Report precisely:** Log browser name, version, OS and screenshots in each defect.

3 DETAILED EXPLANATION [Accuracy & Completeness]

- **Root causes handled:** Differences in rendering engines, unsupported CSS/JS features and vendor-specific behaviour are fixed with standards-based code, polyfills and graceful degradation.
- **Early and continuous testing:** Run browser checks in every sprint, not just before release.
- **Support policy:** Publish a supported-browser list so the scope is clear to customers and teams.
- **Customer impact:** Consistent behaviour reduces abandoned carts and complaints.

4 EXAMPLE [Application & Clarity]

Example: The 'Place Order' button is hidden on Safari because of a CSS flexbox issue. The cross-browser matrix catches it, the layout is fixed with a compatible style rule, and a visual regression test is added.

Conclusion: A risk-based, matrix-driven and partly automated cross-browser strategy ensures a consistent shopping experience on all supported browsers.

Q19

Identifying Validation Gaps in a Logistics Application

Bloom's L3

Question: A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

1 CONCEPT [Conceptual Understanding]

Validation (input/data validation) ensures that data entered into the system is correct, complete and within allowed rules before processing. Improper validation lets invalid data enter the system and produce inconsistent outputs. Testing must deliberately check these controls using both positive and negative test cases.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Identify gaps:** Mandatory fields not enforced, wrong data types or formats (PIN code, phone), weight/dimension limits not checked, invalid dates (delivery before pickup), duplicate consignment numbers, cross-field rules ignored.
- **Design tests:** Equivalence partitioning (valid/invalid classes), boundary value analysis for weight and quantity, negative tests, cross-field and business-rule tests.
- **Verify error handling:** Meaningful messages, no crash, no partial data saved.
- **Test across layers:** Client-side and server-side validation, and API/database constraints.
- **Retest and regress:** Re-run validation tests after every change.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Sample validation test conditions for a shipment form

Field	Rule	Invalid test data	Expected result
PIN code	6 digits	56001 / 56001A	Rejected with message
Weight (kg)	0.1 – 1000	0, 1001, -5	Rejected
Delivery date	≥ pickup date	Before pickup	Rejected
Consignment no.	Unique	Existing number	Duplicate error

- **Early detection:** Validation tests find data problems before they corrupt downstream processing.
- **Reliability:** Consistent, clean data gives consistent outputs and fewer failures.
- **Security and integrity:** Blocks invalid or malicious input.
- **Customer trust:** Accurate tracking and billing.

4 EXAMPLE [Application & Clarity]

Example: A shipment with weight '-20' was accepted and gave a negative freight charge. A BVA/negative test on weight would have found it early.

Conclusion: Systematic validation testing closes input-handling gaps, so the logistics system produces consistent outputs and becomes more reliable.

Question: A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

1 CONCEPT [Conceptual Understanding]

Systematic test design means test cases are deliberately derived from requirements using recognised techniques and documented with complete details; testing as an engineering activity requires planning, design, execution and evaluation with defined deliverables. Random testing has no coverage guarantee and misses defects.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Step 1 – Analyse test basis:** Study requirements and identify test conditions.
- **Step 2 – Choose techniques:** EP, BVA, decision table, state transition, use-case testing and error guessing.
- **Step 3 – Write test cases:** ID, objective, preconditions, steps, input data, expected result, postconditions.
- **Step 4 – Review:** Peer-review test cases for completeness and correctness.
- **Step 5 – Execute and record:** Run the cases and record actual results and defects.
- **Step 6 – Evaluate:** Measure coverage and decide on additional tests.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Sample structured test case

Field	Content
Test case ID	TC_LOGIN_005
Requirement	REQ-LOGIN-02 (password length 8–16)
Precondition	User is registered and on login page
Steps / Input	Enter valid user ID and 8-character password; click Login
Expected result	User logs in and lands on dashboard
Actual / Status	Recorded during execution (Pass/Fail)

- **Higher defect detection:** Techniques target the places where defects hide.
- **Coverage and traceability:** Every requirement has at least one test case.
- **Reusability:** Documented tests can be reused for regression and training.
- **Objectivity and efficiency:** Results do not depend on the individual tester and effort is not wasted on duplicates.
- **Better estimation:** Number and effort of tests can be planned.

4 EXAMPLE [Application & Clarity]

Example: Random clicking on a registration form may verify only the happy path; a systematic design adds blank, over-length, special-character and duplicate-email tests that reveal validation defects.

Conclusion: Systematic test design converts testing from guesswork into an engineering process and is essential for effective defect detection.

Question: A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

1 CONCEPT [Conceptual Understanding]

Data inconsistency between modules (for example registration, laboratory, pharmacy and billing) usually arises at integration points where data is exchanged, transformed or synchronised. Integration and end-to-end testing based on testing principles (early testing, defect clustering at interfaces, context-dependence of healthcare) help to find and eliminate the causes.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Interface mismatch:** Different data types, formats, units (mg vs g), code sets or field lengths between modules.
- **Mapping/transformation errors:** Incorrect conversion when data moves from one module to another.
- **Transaction and synchronisation issues:** Updates committed in one module but not in another; timing and concurrency problems.
- **Duplicate or master-data problems:** Same patient stored with different IDs.
- **Version and configuration differences:** Modules built against different interface versions.
- **Analysis method:** Trace the data flow of a record, compare values at each interface, and review logs and database states.

3 DETAILED EXPLANATION [Accuracy & Completeness]

- **Integration testing:** Verify every interface with valid, invalid and boundary data.
- **Data-flow and CRUD testing:** Create, read, update, delete a record in one module and verify in all others.
- **Database/reconciliation checks:** Compare record counts and values between source and target systems.
- **End-to-end scenarios:** Patient registration → diagnosis → prescription → billing.
- **Negative and failure testing:** Interrupt a transaction to confirm rollback and no partial updates.
- **Regression:** Re-run integration tests after every interface change.

4 EXAMPLE [Application & Clarity]

Example: A patient's allergy updated in the clinical module was not reflected in the pharmacy module because of a failed synchronisation. A cross-module test 'update allergy and check pharmacy view' finds this immediately and protects patient safety.

Conclusion: Methodical integration and data-consistency testing identifies interface-level root causes and ensures that all modules present the same, correct patient data.

Q22

Role of Testers across SDLC Phases and Benefits of Early Involvement

Bloom's L3

Question: A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

1 CONCEPT [Conceptual Understanding]

Testers contribute in every phase of the SDLC, not only during the test phase. Early involvement lets quality activities run in parallel with development and prevents the delays that arise when defects are discovered late.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Identify delay cause:** Testers joined after coding, so requirement and design defects were found late, leading to rework and schedule slips.
- **Define roles per phase:** See the table below.
- **Improve collaboration:** Joint reviews, shared tools, common definition of done and regular meetings.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Tester's role in each SDLC phase

SDLC Phase	Tester's contribution
Requirements	Review requirements, check testability, raise queries, define acceptance criteria
Design	Review design, plan test strategy, design system/integration tests, identify risks
Coding	Prepare test cases and data, set up environment, support unit testing, review code/checklists
Testing	Execute tests, log and verify defects, regression, report status
Deployment	Smoke/sanity tests, release readiness and acceptance support
Maintenance	Regression and confirmation testing of fixes and enhancements; analyse production defects

- **Early defect detection:** Defects in documents are removed before they become code.
- **Less rework and schedule delay:** Test design and environment are ready when code arrives.
- **Better quality:** Clear requirements, testable design and wider review coverage.
- **Team understanding:** Testers know the product and can give better risk-based advice.

4 EXAMPLE [Application & Clarity]

Example: Because the tester prepared test cases during the design phase, an unhandled error scenario was found in the design review, saving a week of rework that would have occurred in system testing.

Conclusion: When testers participate across all SDLC phases, defects are found sooner, delays shrink and overall quality improves.

Question: A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

1 CONCEPT [Conceptual Understanding]

Test coverage measures how much of the software (requirements, code, risks, scenarios) has been exercised by tests. A failure in peak usage shows missing non-functional and scenario coverage, and violates the principle that exhaustive testing is impossible and therefore tests must be chosen on the basis of risk.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Identify gaps:** No load, stress, spike or endurance testing; concurrency and resource-contention scenarios (many users booking the same item) missing; operational profile (expected peak users) not defined; infrastructure limits (CPU, memory, connections) never exercised.
- **Apply principles:** Risk-based testing – peak events are high-risk; context-dependence – the system must be tested in its real usage context; early testing – performance requirements should be defined and verified early.
- **Plan additional tests:** Load test at expected peak, stress test beyond peak, spike test for sudden surges, soak test for long durations.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Coverage dimensions that prevent peak-load failures

Coverage type	What it ensures	Peak-usage example
Requirement coverage	Every requirement has tests	Performance requirement: 5,000 users
Risk coverage	High-risk scenarios are tested	Flash-sale checkout
Scenario / operational-profile coverage	Realistic usage mix	70% browse, 20% cart, 10% pay
Non-functional coverage	Load, stress, spike, endurance	10× normal traffic
Code / structural coverage	Statements, branches executed	Error and timeout paths

- **Finds capacity limits before users do:** Bottlenecks and breaking points are identified in testing.
- **Improves resilience:** Teams can tune resources, add caching, queues and auto-scaling.
- **Reduces risk:** Critical scenarios are never left to chance.
- **Measurable:** Coverage metrics show what is and is not tested.

4 EXAMPLE [Application & Clarity]

Example: A ticket-booking site tested only with 50 users crashed at 3,000. A load test at 3,000–5,000 concurrent users would have exposed the database connection limit before release.

Conclusion: Broader, risk-based test coverage including load and concurrency scenarios prevents peak-time failures and builds dependable systems.

Question: A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

1 CONCEPT [Conceptual Understanding]

A defect repository (defect-tracking database) is a central store in which every defect is documented with complete information and followed through a defined life cycle. The absence of proper documentation makes tracking, accountability and analysis impossible.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Adopt a defect tracking tool:** Jira, Bugzilla, Azure DevOps or similar, with a standard defect template.
- **Define mandatory fields:** ID, title, description, steps to reproduce, expected vs actual result, environment, severity, priority, module, phase injected/detected, reporter, assignee, status, attachments, resolution, root cause.
- **Define the life cycle:** New → Assigned → Open → Fixed → Retest → Verified → Closed (with Reopened, Rejected, Deferred states).
- **Set rules:** Each defect must be logged, reviewed in triage meetings and updated by the owner.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Typical defect record

Field	Sample entry
Defect ID / Title	DEF-214 / Order total wrong after coupon
Module / Environment	Checkout / QA, Chrome 126
Steps to reproduce	Add item ₹1,200; apply coupon SAVE10; view total
Expected vs Actual	₹1,080 vs ₹1,200
Severity / Priority	Major / High
Phase injected / found	Coding / System testing
Status / Assignee	Fixed / Developer A
Root cause	Discount applied before price update

- **Tracking:** Everyone sees the current status of each defect; none gets lost.
- **Accountability:** Owners and dates are recorded.
- **Analysis:** Defect density, trend, aging and phase-wise injection/detection statistics can be produced.
- **Process improvement:** Root-cause data guides prevention actions and test design.
- **Communication:** Clear reports reduce misunderstanding between testers and developers.

4 EXAMPLE [Application & Clarity]

Example: With documented fields, the manager sees that 40% of open defects belong to the Checkout module and that most were injected in coding – and decides on extra code reviews there.

Conclusion: A well-documented defect repository converts scattered issues into organised data that supports tracking, analysis and continuous improvement of quality.

Q25**Redesigning the Testing Approach for Repeated Banking Transaction Failures**

Bloom's L3

Question: A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

1 CONCEPT [Conceptual Understanding]

Repeated transaction failures indicate that testing is not risk-focused and not systematic. Software testing principles – early testing, defect clustering, risk-based testing, pesticide paradox, and context-dependence – can be used to redesign the approach for high-integrity financial transactions.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Risk analysis:** Rank transaction types (transfers, bill payments, standing orders, interest posting) by business impact and defect history.
- **Early testing:** Review requirements for business rules (limits, fees, cut-off times) and verify them from the start.
- **Test design techniques:** Decision tables for limits and fees, state-transition for transaction life cycle, BVA for amounts, EP for account types.
- **Transaction integrity tests:** Atomicity, consistency, isolation and durability (ACID), rollback, duplicate-request (idempotency), timeout and concurrency tests.
- **Reconciliation checks:** Verify debits equal credits and ledgers tally.
- **Regression and automation:** Automate the stable core transaction suite, and run it for every build.
- **Security/compliance tests:** Authentication, authorisation, audit trail.

3 DETAILED EXPLANATION [Accuracy & Completeness]

- **Reliability:** Critical paths and failure modes are verified by design, not by chance.
- **Prevents repetition:** Each failure creates a new regression test, so the same failure cannot recur.
- **Defect detection:** Techniques target boundaries, combinations and states where transaction defects hide.
- **Confidence and compliance:** Documented evidence and consistent execution satisfy audits.

4 EXAMPLE [Application & Clarity]

Example: A repeated 'double debit' failure on retry is addressed with an idempotency test: send the same transfer request twice and verify that only one debit occurs; this test is automated in every regression run.

Conclusion: A principle-based, risk-focused and structured test approach makes banking transactions dependable and prevents repeat failures.

Question: A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

1 CONCEPT [Conceptual Understanding]

Edge cases are inputs or conditions at the extreme limits of normal operation (minimum, maximum, empty, just inside or outside limits). Testing only normal values leaves these unexplored. Applying the principles that testing shows presence of defects and that exhaustive testing is impossible, the tester selects the most defect-prone inputs – the edges.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Identify ranges and limits:** Numeric ranges, string lengths, dates, list sizes, file sizes.
- **Apply BVA:** Test min-1, min, min+1, max-1, max, max+1.
- **Add other edge cases:** Empty/null input, zero, negative values, very large values, maximum field length, special characters, leap-year dates, time-zone changes, first/last record.
- **Combine with EP:** One representative from each valid and invalid class.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Edge-case test data for an age field that accepts 18 to 60

Test value	Category	Expected result
17	Just below minimum	Rejected
18	Minimum boundary	Accepted
19	Just above minimum	Accepted
59	Just below maximum	Accepted
60	Maximum boundary	Accepted
61	Just above maximum	Rejected
Blank / null / abc / -1	Other edge inputs	Rejected with proper message

- **Targets the likeliest defect locations:** Programmers make off-by-one and wrong-operator errors at boundaries.
- **Efficient:** A small number of well-chosen values finds many defects.
- **Prevents crashes and wrong results:** Edge handling is verified before users meet it.
- **Improves robustness:** The software behaves predictably at its limits.

4 EXAMPLE [Application & Clarity]

Example: A bug where age 60 was rejected (code used < 60) was only revealed by testing exactly 60; testing ages like 25 or 40 would have passed.

Conclusion: Including edge cases and boundary testing significantly increases defect detection and prevents failures that occur at the system's limits.

Question: A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

1 CONCEPT [Conceptual Understanding]

A process-based testing approach defines testing as a set of standard, repeatable steps (plan, design, set-up, execute, report, close) with common templates, entry/exit criteria and metrics. Organisation-wide standards (e.g. IEEE 829 documentation, TMMi practices) ensure that all modules are tested in the same, disciplined way.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Define one test process:** A common life cycle and responsibilities for all modules.
- **Standard templates:** Test plan, test case, defect report and summary report formats.
- **Common guidelines:** Naming, severity/priority definitions, technique selection, coverage targets.
- **Entry/exit criteria:** Uniform conditions for starting and finishing each test level.
- **Reviews and audits:** Peer review of test cases; periodic process audits.
- **Metrics and improvement:** Same metrics (coverage, defect density, defect detection %) for every module; compare and improve.

3 DETAILED EXPLANATION [Accuracy & Completeness]

- **Consistent quality:** Each module is tested to the same depth and rigour.
- **Comparability:** Common metrics make modules and releases comparable.
- **Efficiency:** Reusable templates and guidelines save effort and training time.
- **Less dependence on individuals:** Quality does not vary with who tests.
- **Predictability:** Planning, estimation and release decisions become more reliable.

4 EXAMPLE [Application & Clarity]

Example: Module A had detailed tests while Module B had only informal checks. After introducing a common test case template and a minimum 90% requirement-coverage exit criterion, defect leakage from Module B fell noticeably.

Conclusion: Standardising testing through a defined process removes uneven practices and gives consistent, higher-quality outcomes across all modules.

Question: A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

1 CONCEPT [Conceptual Understanding]

Robustness is the ability of software to handle invalid, unexpected or extreme inputs without crashing or compromising security. Because users and attackers do not always enter 'normal' data, testing must include negative testing, error guessing and security-oriented input tests.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Identify validation scenarios:** Empty fields, wrong data types, over-length text, special characters, Unicode, leading/trailing spaces, duplicate submissions, invalid file types, malformed requests.
- **Use techniques:** Equivalence partitioning for invalid classes, BVA, error guessing, fuzz testing and security tests (SQL injection, XSS).
- **Check responses:** Application shows clear messages, does not crash, does not reveal internal errors and does not save invalid data.
- **Validate at both ends:** Client-side for usability and server-side for security.
- **Add to regression:** Keep these tests permanently in the suite.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Unexpected-input test conditions for a registration form

Input	Test data	Expected behaviour
Name	Blank / 500 characters / <script>alert(1)</script>	Validation error; script not executed
Email	abc@, abc@@x.com	Invalid email message
Age	abc, -5, 999	Rejected
Password	' OR '1'='1	Treated as text; login fails safely
Photo upload	.exe or 100 MB file	Rejected with message

- **Robustness:** The application tolerates bad input gracefully.
- **Security:** Injection and scripting vulnerabilities are detected.
- **Data integrity:** Only valid data is stored.
- **User experience:** Clear, helpful messages instead of crashes or technical errors.

4 EXAMPLE [Application & Clarity]

Example: Entering a 5,000-character name crashed the registration page. A boundary and over-length test finds this, and the fix adds a length check and a friendly error.

Conclusion: Thorough negative and validation testing improves input handling, so the application remains stable, secure and robust under unexpected inputs.

Question: A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

1 CONCEPT [Conceptual Understanding]

Defect leakage is the number of defects that escape testing and are found by users in production. It is measured as: $\text{Defect Leakage \%} = (\text{Defects found in production} \div (\text{Defects found in testing} + \text{Defects found in production})) \times 100$. It is a key indicator of testing effectiveness.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Collect data:** Analyse each production defect: which requirement, module and test level should have caught it.
- **Find root causes:** Use 5-Whys or fishbone analysis.
- **Typical causes:** Missed or ambiguous requirements; inadequate test coverage; poor test case design; no regression testing; test environment different from production; time pressure; late changes without re-testing; weak reviews.
- **Apply principles:** Early testing, risk-based coverage, defect clustering, pesticide paradox (refresh tests), context-dependence (production-like data/environment).

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Root cause → corrective testing practice

Root cause of leakage	Preventive testing practice
Requirement gaps	Requirement reviews; traceability matrix
Weak test design	Use EP, BVA, decision tables; peer review test cases
Environment mismatch	Production-like test environment and data
No regression after changes	Automated, risk-based regression suite
Time pressure	Risk-based prioritisation; clear exit criteria
Repeated escapes	Add a test for every escaped defect

- **Targets the causes:** Fixing causes reduces the number of escapes in future releases.
- **Measurable:** Leakage % is tracked each release, e.g. 4 production defects against 76 found in testing gives 5%; the goal is steady reduction.
- **Increases confidence:** Better coverage and stricter exit criteria give more dependable releases.
- **Lower support cost:** Fewer hot-fixes and customer escalations.

4 EXAMPLE [Application & Clarity]

Example: Analysis showed 60% of leaked defects occurred in configuration-specific scenarios. After adding a production-like staging environment and configuration tests, leakage dropped release by release.

Conclusion: Root-cause-based improvements in test coverage, design, environment and regression reduce defect leakage and protect production quality.

Q30

Early Defect Detection Strategy to Reduce Cost and Effort

Bloom's L3

Question: A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

1 CONCEPT [Conceptual Understanding]

Poor testing strategy that finds defects late increases cost, because the cost of a defect rises steeply the later it is found. A redesigned strategy applying the principle of early testing ('shift-left') integrates verification and validation activities from the very start of the project.

2 APPLICATION TO THE SCENARIO [Application of Knowledge]

- **Static testing first:** Reviews, walkthroughs and inspections of requirements, design and code; static analysis tools.
- **V-model / parallel test planning:** Prepare test plans and cases during each development phase.
- **Unit testing and continuous integration:** Developers run unit tests; every build triggers automated smoke/regression tests.
- **Risk-based prioritisation:** Test high-risk and defect-prone areas first.
- **Clear entry/exit criteria:** Quality gates between phases.
- **Metrics:** Track defect detection percentage per phase and phase containment.

3 DETAILED EXPLANATION [Accuracy & Completeness]

Table: Illustrative relative cost of fixing a defect by phase of detection

Phase where defect is found	Relative cost to fix	Comment
Requirements	1×	Correct a statement
Design	≈ 5×	Change design and documents
Coding / unit test	≈ 10×	Fix code and unit tests
System testing	≈ 20×	Fix, rebuild, retest, regress
Production	≈ 100×	Hot-fix, downtime, reputation loss

- **Lower cost:** The same defect is cheaper to fix in an early phase (figures are illustrative of the commonly cited trend).
- **Less effort and rework:** Fewer defects reach later, more expensive test levels.
- **Faster delivery:** Stable builds and shorter test cycles.
- **Higher efficiency:** Teams spend time on building features rather than fixing late defects.

4 EXAMPLE [Application & Clarity]

Example: A review finds that the SRS lacks a refund rule. Adding one line in the requirements phase costs minutes; finding the gap in production would require design, code and data changes and customer compensation.

Conclusion: A shift-left strategy that detects defects early lowers rework, cost and risk, and improves the overall efficiency of the project.

